Министерство образования Республики Беларусь

Учреждения образования

«Брестский государственный технический университет

Факультет электронно-информационных систем

Кафедра интеллектуальных информационных технологий

**Отчет по лабораторной работе №5**

Специальность ПО-12

Выполнил
К. В. Ковалюк,
студент группы ПО-12

Проверил
А. Д. Кулик,
преп. стаж. кафедры ИИТ,
«___» _____2026 г.

Брест 2026

**Цель работы:** Приобрести практические навыки разработки API и баз данных.

**Вариант: №7**

**Общее задание**

1. Реализовать базу данных из не менее 5 таблиц на заданную тематику. При реализации продумать типизацию полей и внешние ключи в таблицах;

2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);

3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;

4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);

5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпойнт; Базу данные можно реализовать в любой СУБД (MySQL, PostgreSQL, SQLite и др.)

7) База данных Компьютерная лаборатория

**Выполнение:**

**Код программы:**

**create_db.py:**

```
from database import engine, Base, SessionLocal
from crud import create_client, create_manufacturer, create_product, create_computer
from datetime import date

Base.metadata.drop_all(bind=engine)
Base.metadata.create_all(bind=engine)

db = SessionLocal()

client1 = create_client(db, "Иван", "Петров", "M", "ivan_p", "+7(123)456-78-90", "ivan@mail.ru")
client2 = create_client(db, "Мария", "Сидорова", "F", "masha_s", "+7(098)765-43-21", "maria@mail.ru")
client3 = create_client(db, "Алексей", "Иванов", "M", None, "+7(555)123-45-67", "alex@mail.ru")

man1 = create_manufacturer(db, "Intel Corporation", date(1968, 7, 18))
man2 = create_manufacturer(db, "AMD", date(1969, 5, 1))
man3 = create_manufacturer(db, "NVIDIA", date(1993, 4, 5))
man4 = create_manufacturer(db, "ASUS", date(1989, 4, 2))
man5 = create_manufacturer(db, "Kingston", date(1987, 10, 17))

product1 = create_product(db, "Процессор Intel Core i9-13900K", 58990.00, man1.man_id)
product2 = create_product(db, "Процессор AMD Ryzen 9 7950X", 54990.00, man2.man_id)
product3 = create_product(db, "Видеокарта NVIDIA RTX 4090", 189990.00, man3.man_id)
product4 = create_product(db, "Материнская плата ASUS ROG Maximus Z790", 45990.00, man4.man_id)
product5 = create_product(db, "Оперативная память Kingston FURY 32GB", 12990.00, man5.man_id)
product6 = create_product(db, "Видеокарта NVIDIA RTX 4080", 129990.00, man3.man_id)
product7 = create_product(db, "Процессор Intel Core i7-13700K", 38990.00, man1.man_id)

computer1 = create_computer(db, "SN001", "Gaming PC Ultra", "Intel Core i9-13900K", 32, 1000,
product1.pr_id, "available")
computer2 = create_computer(db, "SN002", "Workstation Pro", "AMD Ryzen 9 7950X", 64, 2000,
product2.pr_id, "available")
computer3 = create_computer(db, "SN003", "Gaming PC RTX", "Intel Core i7-13700K", 32, 1000,
product6.pr_id, "rented")
computer4 = create_computer(db, "SN004", "Design Station", "AMD Ryzen 9 7950X", 128, 4000,
product3.pr_id, "maintenance")

print("База данных успешно создана и заполнена тестовыми данными!")
print(f"Добавлено клиентов: 3")
print(f"Добавлено производителей: 5")
print(f"Добавлено продуктов: 7")
print(f"Добавлено компьютеров: 4")
```

```
db.close()
```

**crud.py:**

```
from sqlalchemy.orm import Session
from datetime import date
from database import Client, Manufacturer, Product, Order, OrdersSummary, Computer

def create_client(db: Session, first_name, last_name, sex, nick_name=None, phone=None, email=None):
    client = Client(
        first_name=first_name,
        last_name=last_name,
        sex=sex,
        nick_name=nick_name,
        phone=phone,
        email=email
    )
    db.add(client)
    db.commit()
    db.refresh(client)
    return client

def get_clients(db: Session, skip=0, limit=100):
    return db.query(Client).offset(skip).limit(limit).all()

def get_client(db: Session, client_id):
    return db.query(Client).filter(Client.d_id == client_id).first()

def update_client(db: Session, client_id, **kwargs):
    client = db.query(Client).filter(Client.d_id == client_id).first()
    if client:
        for key, value in kwargs.items():
            if hasattr(client, key):
                setattr(client, key, value)
        db.commit()
        db.refresh(client)
    return client

def delete_client(db: Session, client_id):
    client = db.query(Client).filter(Client.d_id == client_id).first()
    if client:
        db.delete(client)
        db.commit()
        return True
    return False

def create_product(db: Session, name, price, man_id):
    product = Product(name=name, price=price, man_id=man_id)
    db.add(product)
    db.commit()
    db.refresh(product)
    return product

def get_products(db: Session, skip=0, limit=100):
    return db.query(Product).offset(skip).limit(limit).all()

def get_product(db: Session, product_id):
    return db.query(Product).filter(Product.pr_id == product_id).first()

def update_product(db: Session, product_id, **kwargs):
    product = db.query(Product).filter(Product.pr_id == product_id).first()
    if product:
        for key, value in kwargs.items():
            if hasattr(product, key):
                setattr(product, key, value)
        db.commit()
        db.refresh(product)
```

```
    return product

def delete_product(db: Session, product_id):
    product = db.query(Product).filter(Product.pr_id == product_id).first()
    if product:
        db.delete(product)
        db.commit()
        return True
    return False

def create_order(db: Session, d_id, items):
    order = Order(d_id=d_id, date_of_order=date.today(), total_sum=0.00)
    db.add(order)
    db.flush()

    total = 0
    for item in items:
        product = db.query(Product).filter(Product.pr_id == item["pr_id"]).first()
        if product:
            order_item = OrdersSummary(
                ord_id=order.ord_id,
                pr_id=item["pr_id"],
                count=item["count"]
            )
            db.add(order_item)
            total += float(product.price) * item["count"]

    order.total_sum = total
    db.commit()
    db.refresh(order)
    return order

def get_orders(db: Session, skip=0, limit=100):
    return db.query(Order).offset(skip).limit(limit).all()

def get_order(db: Session, order_id):
    return db.query(Order).filter(Order.ord_id == order_id).first()

def update_order(db: Session, order_id, **kwargs):
    order = db.query(Order).filter(Order.ord_id == order_id).first()
    if order:
        for key, value in kwargs.items():
            if hasattr(order, key):
                setattr(order, key, value)
        db.commit()
        db.refresh(order)
    return order

def delete_order(db: Session, order_id):
    order = db.query(Order).filter(Order.ord_id == order_id).first()
    if order:
        db.query(OrdersSummary).filter(OrdersSummary.ord_id == order_id).delete()
        db.delete(order)
        db.commit()
        return True
    return False

def create_manufacturer(db: Session, name, establish_date):
    manufacturer = Manufacturer(name=name, establish_date=establish_date)
    db.add(manufacturer)
    db.commit()
    db.refresh(manufacturer)
    return manufacturer

def get_manufacturers(db: Session, skip=0, limit=100):
    return db.query(Manufacturer).offset(skip).limit(limit).all()
```

```
def get_manufacturer(db: Session, man_id):
    return db.query(Manufacturer).filter(Manufacturer.man_id == man_id).first()

def update_manufacturer(db: Session, man_id, **kwargs):
    manufacturer = db.query(Manufacturer).filter(Manufacturer.man_id == man_id).first()
    if manufacturer:
        for key, value in kwargs.items():
            if hasattr(manufacturer, key):
                setattr(manufacturer, key, value)
        db.commit()
        db.refresh(manufacturer)
    return manufacturer

def delete_manufacturer(db: Session, man_id):
    manufacturer = db.query(Manufacturer).filter(Manufacturer.man_id == man_id).first()
    if manufacturer:
        db.delete(manufacturer)
        db.commit()
        return True
    return False

def get_order_details(db: Session, order_id):
    result = db.query(
        Order.ord_id,
        Client.first_name,
        Client.last_name,
        Order.date_of_order,
        Order.total_sum,
        Product.name,
        OrdersSummary.count,
        (Product.price * OrdersSummary.count).label("subtotal")
    ).join(Client, Order.d_id == Client.d_id)\
     .join(OrdersSummary, Order.ord_id == OrdersSummary.ord_id)\
     .join(Product, OrdersSummary.pr_id == Product.pr_id)\
     .filter(Order.ord_id == order_id).all()
    return result

def create_computer(db: Session, serial_number, model, processor, ram_gb, storage_gb, pr_id,
status="available"):
    computer = Computer(
        serial_number=serial_number,
        model=model,
        processor=processor,
        ram_gb=ram_gb,
        storage_gb=storage_gb,
        status=status,
        pr_id=pr_id
    )
    db.add(computer)
    db.commit()
    db.refresh(computer)
    return computer

def get_computers(db: Session, skip=0, limit=100):
    return db.query(Computer).offset(skip).limit(limit).all()

def get_computer(db: Session, comp_id):
    return db.query(Computer).filter(Computer.comp_id == comp_id).first()

def update_computer(db: Session, comp_id, **kwargs):
    computer = db.query(Computer).filter(Computer.comp_id == comp_id).first()
    if computer:
        for key, value in kwargs.items():
            if hasattr(computer, key):
                setattr(computer, key, value)
```

```
        db.commit()
        db.refresh(computer)
    return computer

def delete_computer(db: Session, comp_id):
    computer = db.query(Computer).filter(Computer.comp_id == comp_id).first()
    if computer:
        db.delete(computer)
        db.commit()
        return True
    return False
```

## database.py:

```
from sqlalchemy import create_engine, Column, Integer, String, DECIMAL, Date, ForeignKey
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, relationship

Base = declarative_base()

class Client(Base):
    __tablename__ = "clients"

    d_id = Column(Integer, primary_key=True)
    first_name = Column(String(50), nullable=False)
    last_name = Column(String(50), nullable=False)
    sex = Column(String(1), nullable=False)
    nick_name = Column(String(50))
    phone = Column(String(20))
    email = Column(String(30))

    orders = relationship("Order", back_populates="client")

class Manufacturer(Base):
    __tablename__ = "manufacturers"

    man_id = Column(Integer, primary_key=True)
    name = Column(String(100), nullable=False)
    establish_date = Column(Date, nullable=False)

    products = relationship("Product", back_populates="manufacturer")

class Product(Base):
    __tablename__ = "products"

    pr_id = Column(Integer, primary_key=True)
    name = Column(String(100), nullable=False)
    price = Column(DECIMAL(15, 2), nullable=False)
    man_id = Column(Integer, ForeignKey("manufacturers.man_id"), nullable=False)

    manufacturer = relationship("Manufacturer", back_populates="products")
    order_items = relationship("OrdersSummary", back_populates="product")

class Order(Base):
    __tablename__ = "orders"

    ord_id = Column(Integer, primary_key=True)
    d_id = Column(Integer, ForeignKey("clients.d_id"), nullable=False)
    date_of_order = Column(Date, nullable=False)
    total_sum = Column(DECIMAL(15, 2), default=0.00)

    client = relationship("Client", back_populates="orders")
    items = relationship("OrdersSummary", back_populates="order")

class OrdersSummary(Base):
    __tablename__ = "orders_summary"
```

```
    ord_s_id = Column(Integer, primary_key=True)
    ord_id = Column(Integer, ForeignKey("orders.ord_id"), nullable=False)
    pr_id = Column(Integer, ForeignKey("products.pr_id"), nullable=False)
    count = Column(Integer, nullable=False)

    order = relationship("Order", back_populates="items")
    product = relationship("Product", back_populates="order_items")

class Computer(Base):
    __tablename__ = "computers"

    comp_id = Column(Integer, primary_key=True)
    serial_number = Column(String(50), nullable=False, unique=True)
    model = Column(String(100), nullable=False)
    processor = Column(String(100))
    ram_gb = Column(Integer)
    storage_gb = Column(Integer)
    status = Column(String(20), default="available")
    pr_id = Column(Integer, ForeignKey("products.pr_id"), nullable=False)

    product = relationship("Product")

engine = create_engine("sqlite:///computer_lab.db", echo=True)
SessionLocal = sessionmaker(bind=engine)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

**main.py:**

```
from fastapi import FastAPI, Depends, HTTPException
from sqlalchemy.orm import Session
from typing import List, Optional
from pydantic import BaseModel
from datetime import date

from database import get_db, engine, Base
import crud

app = FastAPI(title="Computer Laboratory API")

Base.metadata.create_all(bind=engine)

class ClientCreate(BaseModel):
    first_name: str
    last_name: str
    sex: str
    nick_name: Optional[str] = None
    phone: Optional[str] = None
    email: Optional[str] = None

class ClientResponse(BaseModel):
    d_id: int
    first_name: str
    last_name: str
    sex: str
    nick_name: Optional[str] = None
    phone: Optional[str] = None
    email: Optional[str] = None

class ProductCreate(BaseModel):
    name: str
    price: float
```

```
    man_id: int

class ProductResponse(BaseModel):
    pr_id: int
    name: str
    price: float
    man_id: int

class OrderItem(BaseModel):
    pr_id: int
    count: int

class OrderCreate(BaseModel):
    d_id: int
    items: List[OrderItem]

class OrderResponse(BaseModel):
    ord_id: int
    d_id: int
    date_of_order: date
    total_sum: float

class ManufacturerCreate(BaseModel):
    name: str
    establish_date: date

class ManufacturerResponse(BaseModel):
    man_id: int
    name: str
    establish_date: date

class ComputerCreate(BaseModel):
    serial_number: str
    model: str
    processor: Optional[str] = None
    ram_gb: Optional[int] = None
    storage_gb: Optional[int] = None
    pr_id: int
    status: Optional[str] = "available"

class ComputerResponse(BaseModel):
    comp_id: int
    serial_number: str
    model: str
    processor: Optional[str] = None
    ram_gb: Optional[int] = None
    storage_gb: Optional[int] = None
    status: str
    pr_id: int

@app.get("/")
def read_root():
    return {"message": "Computer Laboratory API"}

@app.post("/clients/", response_model=ClientResponse)
def create_client(client: ClientCreate, db: Session = Depends(get_db)):
    return crud.create_client(
        db=db,
        first_name=client.first_name,
        last_name=client.last_name,
        sex=client.sex,
        nick_name=client.nick_name,
        phone=client.phone,
        email=client.email
    )
```

```
@app.get("/clients/", response_model=List[ClientResponse])
def read_clients(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    clients = crud.get_clients(db, skip=skip, limit=limit)
    return clients

@app.get("/clients/{client_id}", response_model=ClientResponse)
def read_client(client_id: int, db: Session = Depends(get_db)):
    client = crud.get_client(db, client_id)
    if client is None:
        raise HTTPException(status_code=404, detail="Client not found")
    return client

@app.put("/clients/{client_id}", response_model=ClientResponse)
def update_client(client_id: int, client: ClientCreate, db: Session = Depends(get_db)):
    updated = crud.update_client(
        db=db,
        client_id=client_id,
        first_name=client.first_name,
        last_name=client.last_name,
        sex=client.sex,
        nick_name=client.nick_name,
        phone=client.phone,
        email=client.email
    )
    if updated is None:
        raise HTTPException(status_code=404, detail="Client not found")
    return updated

@app.delete("/clients/{client_id}")
def delete_client(client_id: int, db: Session = Depends(get_db)):
    success = crud.delete_client(db, client_id)
    if not success:
        raise HTTPException(status_code=404, detail="Client not found")
    return {"message": "Client deleted successfully"}

@app.post("/products/", response_model=ProductResponse)
def create_product(product: ProductCreate, db: Session = Depends(get_db)):
    return crud.create_product(db=db, name=product.name, price=product.price, man_id=product.man_id)

@app.get("/products/", response_model=List[ProductResponse])
def read_products(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    products = crud.get_products(db, skip=skip, limit=limit)
    return products

@app.get("/products/{product_id}", response_model=ProductResponse)
def read_product(product_id: int, db: Session = Depends(get_db)):
    product = crud.get_product(db, product_id)
    if product is None:
        raise HTTPException(status_code=404, detail="Product not found")
    return product

@app.put("/products/{product_id}", response_model=ProductResponse)
def update_product(product_id: int, product: ProductCreate, db: Session = Depends(get_db)):
    updated = crud.update_product(
        db=db,
        product_id=product_id,
        name=product.name,
        price=product.price,
        man_id=product.man_id
    )
    if updated is None:
        raise HTTPException(status_code=404, detail="Product not found")
    return updated

@app.delete("/products/{product_id}")
def delete_product(product_id: int, db: Session = Depends(get_db)):
```

```
    success = crud.delete_product(db, product_id)
    if not success:
        raise HTTPException(status_code=404, detail="Product not found")
    return {"message": "Product deleted successfully"}

@app.post("/orders/", response_model=OrderResponse)
def create_order(order: OrderCreate, db: Session = Depends(get_db)):
    items = [{"pr_id": item.pr_id, "count": item.count} for item in order.items]
    return crud.create_order(db=db, d_id=order.d_id, items=items)

@app.get("/orders/", response_model=List[OrderResponse])
def read_orders(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    orders = crud.get_orders(db, skip=skip, limit=limit)
    return orders

@app.get("/orders/{order_id}", response_model=OrderResponse)
def read_order(order_id: int, db: Session = Depends(get_db)):
    order = crud.get_order(db, order_id)
    if order is None:
        raise HTTPException(status_code=404, detail="Order not found")
    return order

@app.delete("/orders/{order_id}")
def delete_order(order_id: int, db: Session = Depends(get_db)):
    success = crud.delete_order(db, order_id)
    if not success:
        raise HTTPException(status_code=404, detail="Order not found")
    return {"message": "Order deleted successfully"}

@app.post("/manufacturers/", response_model=ManufacturerResponse)
def create_manufacturer(manufacturer: ManufacturerCreate, db: Session = Depends(get_db)):
    return crud.create_manufacturer(db=db, name=manufacturer.name,
establish_date=manufacturer.establish_date)

@app.get("/manufacturers/", response_model=List[ManufacturerResponse])
def read_manufacturers(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    manufacturers = crud.get_manufacturers(db, skip=skip, limit=limit)
    return manufacturers

@app.get("/manufacturers/{man_id}", response_model=ManufacturerResponse)
def read_manufacturer(man_id: int, db: Session = Depends(get_db)):
    manufacturer = crud.get_manufacturer(db, man_id)
    if manufacturer is None:
        raise HTTPException(status_code=404, detail="Manufacturer not found")
    return manufacturer

@app.put("/manufacturers/{man_id}", response_model=ManufacturerResponse)
def update_manufacturer(man_id: int, manufacturer: ManufacturerCreate, db: Session = Depends(get_db)):
    updated = crud.update_manufacturer(
        db=db,
        man_id=man_id,
        name=manufacturer.name,
        establish_date=manufacturer.establish_date
    )
    if updated is None:
        raise HTTPException(status_code=404, detail="Manufacturer not found")
    return updated

@app.delete("/manufacturers/{man_id}")
def delete_manufacturer(man_id: int, db: Session = Depends(get_db)):
    success = crud.delete_manufacturer(db, man_id)
    if not success:
        raise HTTPException(status_code=404, detail="Manufacturer not found")
    return {"message": "Manufacturer deleted successfully"}

@app.get("/orders/{order_id}/details")
```

```
def get_order_details(order_id: int, db: Session = Depends(get_db)):
    details = crud.get_order_details(db, order_id)
    if not details:
        raise HTTPException(status_code=404, detail="Order not found")
    return details

@app.post("/computers/", response_model=ComputerResponse)
def create_computer(computer: ComputerCreate, db: Session = Depends(get_db)):
    return crud.create_computer(
        db=db,
        serial_number=computer.serial_number,
        model=computer.model,
        processor=computer.processor,
        ram_gb=computer.ram_gb,
        storage_gb=computer.storage_gb,
        pr_id=computer.pr_id,
        status=computer.status
    )

@app.get("/computers/", response_model=List[ComputerResponse])
def read_computers(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    computers = crud.get_computers(db, skip=skip, limit=limit)
    return computers

@app.get("/computers/{comp_id}", response_model=ComputerResponse)
def read_computer(comp_id: int, db: Session = Depends(get_db)):
    computer = crud.get_computer(db, comp_id)
    if computer is None:
        raise HTTPException(status_code=404, detail="Computer not found")
    return computer

@app.put("/computers/{comp_id}", response_model=ComputerResponse)
def update_computer(comp_id: int, computer: ComputerCreate, db: Session = Depends(get_db)):
    updated = crud.update_computer(
        db=db,
        comp_id=comp_id,
        serial_number=computer.serial_number,
        model=computer.model,
        processor=computer.processor,
        ram_gb=computer.ram_gb,
        storage_gb=computer.storage_gb,
        pr_id=computer.pr_id,
        status=computer.status
    )
    if updated is None:
        raise HTTPException(status_code=404, detail="Computer not found")
    return updated

@app.delete("/computers/{comp_id}")
def delete_computer(comp_id: int, db: Session = Depends(get_db)):
    success = crud.delete_computer(db, comp_id)
    if not success:
        raise HTTPException(status_code=404, detail="Computer not found")
    return {"message": "Computer deleted successfully"}
```

**run.py:**

```
import uvicorn
if __name__ == "__main__":
    uvicorn.run("main:app", host="127.0.0.1", port=8000, reload=True)
```

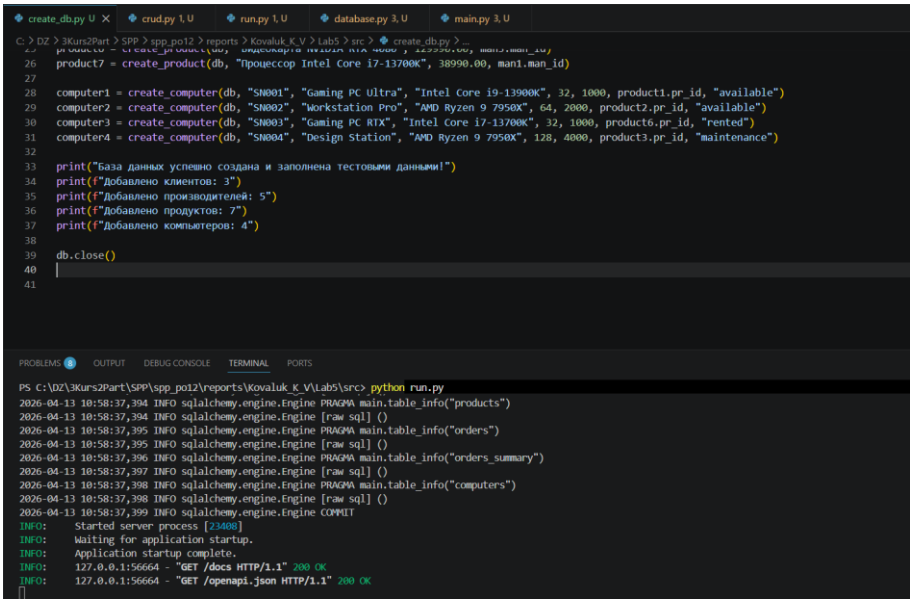
**requirements.txt**

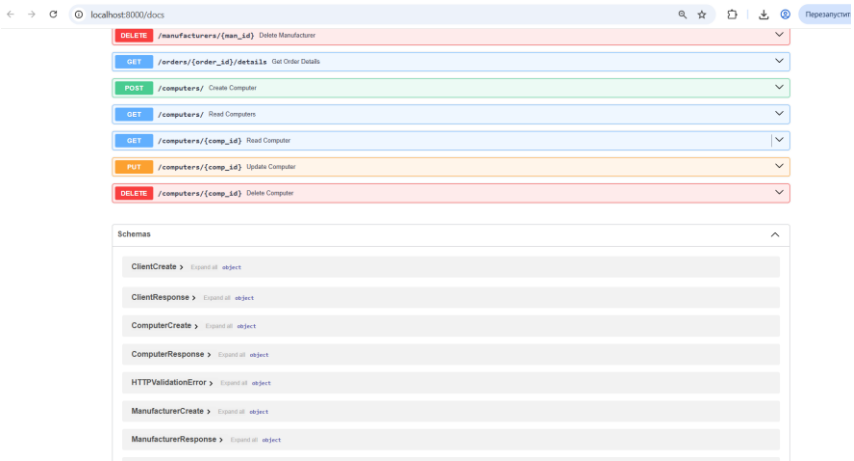
```
fastapi==0.115.0
uvicorn==0.30.0
sqlalchemy==2.0.35
```

**Результат работы программы:**

*PS C:\DZ\3Kurs2Part\SPP\spp_po12\reports\Kovaluk_K_V\Lab5\src> python run.py*
*INFO: Will watch for changes in these directories: ['C:\\DZ\\3Kurs2Part\\SPP\\spp_po12\\reports\\Kovaluk_K_V\\Lab5\\src']*
*INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)*
*INFO: Started reloader process [26956] using StatReload*
*2026-04-13 10:58:37,389 INFO sqlalchemy.engine.Engine BEGIN (implicit)*
*2026-04-13 10:58:37,390 INFO sqlalchemy.engine.Engine PRAGMA main.table_info("clients")*
*2026-04-13 10:58:37,390 INFO sqlalchemy.engine.Engine [raw sql] ()*
*2026-04-13 10:58:37,392 INFO sqlalchemy.engine.Engine PRAGMA main.table_info("manufacturers")*
*2026-04-13 10:58:37,393 INFO sqlalchemy.engine.Engine [raw sql] ()*
*2026-04-13 10:58:37,394 INFO sqlalchemy.engine.Engine PRAGMA main.table_info("products")*
*2026-04-13 10:58:37,394 INFO sqlalchemy.engine.Engine [raw sql] ()*
*2026-04-13 10:58:37,395 INFO sqlalchemy.engine.Engine PRAGMA main.table_info("orders")*
*2026-04-13 10:58:37,395 INFO sqlalchemy.engine.Engine [raw sql] ()*
*2026-04-13 10:58:37,396 INFO sqlalchemy.engine.Engine PRAGMA main.table_info("orders_summary")*
*2026-04-13 10:58:37,397 INFO sqlalchemy.engine.Engine [raw sql] ()*
*2026-04-13 10:58:37,398 INFO sqlalchemy.engine.Engine PRAGMA main.table_info("computers")*
*2026-04-13 10:58:37,398 INFO sqlalchemy.engine.Engine [raw sql] ()*
*2026-04-13 10:58:37,399 INFO sqlalchemy.engine.Engine COMMIT*
*INFO: Started server process [23408]*
*INFO: Waiting for application startup.*
*INFO: Application startup complete.*
*INFO: 127.0.0.1:56664 - "GET /docs HTTP/1.1" 200 OK*
*INFO: 127.0.0.1:56664 - "GET /openapi.json HTTP/1.1" 200 OK*

**Рисунок с результатом работы программы:**




**Вывод:** В ходе работы я приобрел практические навыки разработки API и баз данных.